Leçon 3 : Logique du Premier Ordre

Quantificateurs et Égalité dans Rocq

Logique et Fondements de l'Informatique

Année 2026

Bonjour à tous. Aujourd'hui nous entamons la troisième leçon. Jusqu'à présent, nous étions cantonnés à la logique propositionnelle, où les atomes étaient de simples variables A, B, C invisibles de l'intérieur. Aujourd'hui, on ouvre le capot et on regarde les individus qui peuplent nos mondes mathématiques avec la logique des prédicats.

Les Prédicats comme Fonctions

D'une proposition à une propriété :

- Avant : A : Prop

Les Prédicats comme Fonctions
D'une proposition à une propriété :
Avant : A : Prop

Dans Rocq, et par l'inspiration originale de Alonzo Church, un prédicat n'est rien d'autre qu'une fonction. Elle prend un individu et recrache une proposition. C'est une vision très élégante où les λ-termes servent à la fois pour les preuves et pour les prédicats eux-mêmes. Attention cependant, dès qu'on touche à un univers de type, on doit faire attention au fameux paradoxe de Girard, qui est l'analogue du paradoxe de Russell pour les types de types. Heureusement l'architecture de Rocq gère ça via les univers.

Les Prédicats comme Fonctions

D'une proposition à une propriété :

- Avant : A : Prop
- Maintenant : Type domaine T
- Prédicat : $P : T \to \text{Prop}$

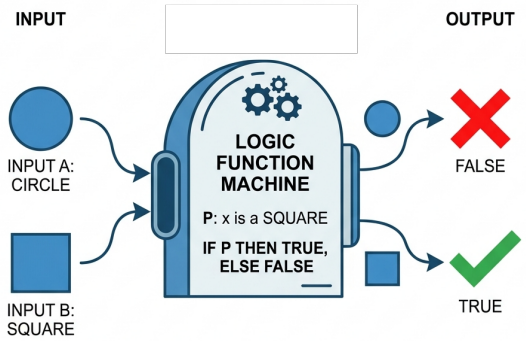
Les Prédicats comme Fonctions
D'une proposition à une propriété :
Avant : A : Prop
Maintenant : Type domaine T
Prédicat : $P : T \to \text{Prop}$

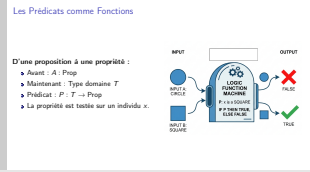
Dans Rocq, et par l'inspiration originale de Alonzo Church, un prédicat n'est rien d'autre qu'une fonction. Elle prend un individu et recrache une proposition. C'est une vision très élégante où les λ-termes servent à la fois pour les preuves et pour les prédicats eux-mêmes. Attention cependant, dès qu'on touche à un univers de type, on doit faire attention au fameux paradoxe de Girard, qui est l'analogue du paradoxe de Russell pour les types de types. Heureusement l'architecture de Rocq gère ça via les univers.

Les Prédicats comme Fonctions

D'une proposition à une propriété :

- Avant : A : Prop
- Maintenant : Type domaine T
- Prédicat : $P : T \to \text{Prop}$
- La propriété est testée sur un individu x.





Dans Rocq, et par l'inspiration originale de Alonzo Church, un prédicat n'est rien d'autre qu'une fonction. Elle prend un individu et recrache une proposition. C'est une vision très élégante où les λ-termes servent à la fois pour les preuves et pour les prédicats eux-mêmes. Attention cependant, dès qu'on touche à un univers de type, on doit faire attention au fameux paradoxe de Girard, qui est l'analogue du paradoxe de Russell pour les types de types. Heureusement l'architecture de Rocq gère ça via les univers.

Le Quantificateur Universel (∀)

- Noté `forall x : T, P x` en Rocq.

Le Quantificateur Universel (∀)
Noté forall x : T, P x en Rocq.

Il est crucial de comprendre que 'A -> B' et 'forall x : A, ...' sont manipulés exactement par les mêmes tactiques. Une implication c'est juste un pour tout qui ignore l'individu en argument ! C'est la beauté de la correspondance de Curry-Howard.

Le Quantificateur Universel (∀)

- Noté `forall x : T, P x` en Rocq.
- Informatiquement : un **type dépendant**.

Le Quantificateur Universel (∀)
Noté forall x : T, P x en Rocq.
Informatiquement : un type dépendant.

Il est crucial de comprendre que ‘A -> B‘ et ‘forall x : A, ...‘ sont manipulés exactement par les mêmes tactiques. Une implication c'est juste un pour tout qui ignore l'individu en argument ! C'est la beauté de la correspondance de Curry-Howard.

Le Quantificateur Universel (∀)

- Noté `forall x : T, P x` en Rocq.
- Informatiquement : un **type dépendant**.
- C'est une fonction qui à tout x associe une *preuve* de $P(x)$.

Le Quantificateur Universel (∀)
Noté forall x : T, P x en Rocq.
Informatiquement : un type dépendant.
C'est une fonction qui à tout x associe une preuve de P(x).

Il est crucial de comprendre que 'A -> B' et 'forall x : A, ...' sont manipulés exactement par les mêmes tactiques. Une implication c'est juste un pour tout qui ignore l'individu en argument ! C'est la beauté de la correspondance de Curry-Howard.

Le Quantificateur Universel (∀)

- Noté `forall x : T, P x` en Rocq.
- Informatiquement : un **type dépendant**.
- C'est une fonction qui à tout x associe une *preuve* de $P(x)$.
- L'implication `A -> B` est juste un cas particulier !

Le Quantificateur Universel (∀)
- Noté `forall x : T, P x` en Rocq.
- Informatiquement : un **type dépendant**.
- C'est une fonction qui à tout x associe une *preuve* de $P(x)$.
- L'implication `A -> B` est juste un cas particulier !

Il est crucial de comprendre que 'A -> B' et 'forall x : A, ...' sont manipulés exactement par les mêmes tactiques. Une implication c'est juste un pour tout qui ignore l'individu en argument ! C'est la beauté de la correspondance de Curry-Howard.

Le Quantificateur Existentiel (∃)

- Noté `exists x : T, P x` en Rocq.

Le Quantificateur Existentiel (∃)
Noté exists x : T, P x en Rocq.

En constructivisme strict, dire "Il existe", c'est être capable de montrer "Voici la chose". C'est pour ça qu'on a besoin d'exhiber le témoin physiquement avec la tactique ‘exists‘. Et quand on a une hypothèse existentielle, ‘destruct‘ nous la coupe en deux : le témoin, et sa preuve.

Le Quantificateur Existentiel (∃)

- Noté `exists x : T, P x` en Rocq.
- Constructivisme exige : il faut trouver *le témoin*.

Le Quantificateur Existentiel (∃)
- Noté `exists x : T, P x` en Rocq.
- Constructivisme exige : il faut trouver *le témoin*.

En constructivisme strict, dire "Il existe", c'est être capable de montrer "Voici la chose". C'est pour ça qu'on a besoin d'exhiber le témoin physiquement avec la tactique ‘exists‘. Et quand on a une hypothèse existentielle, ‘destruct‘ nous la coupe en deux : le témoin, et sa preuve.

Le Quantificateur Existentiel (∃)

- Noté `exists x : T, P x` en Rocq.
- Constructivisme exige : il faut trouver *le témoin*.
- Preuve = Paire (t, π)
  - $t : T$ (le témoin)
  - $\pi : P(t)$ (la justification)

Le Quantificateur Existentiel (∃)
Noté exists x : T, P x en Rocq.
Constructivisme exige : il faut trouver le témoin.
Preuve = Paire (t, π)
t : T (le témoin)
π : P(t) (la justification)

En constructivisme strict, dire "Il existe", c'est être capable de montrer "Voici la chose". C'est pour ça qu'on a besoin d'exhiber le témoin physiquement avec la tactique ‘exists‘. Et quand on a une hypothèse existentielle, ‘destruct‘ nous la coupe en deux : le témoin, et sa preuve.

Le Quantificateur Existentiel (∃)

- Noté `exists x : T, P x` en Rocq.
- Constructivisme exige : il faut trouver *le témoin*.
- Preuve = Paire (t, π)
  - $t : T$ (le témoin)
  - $\pi : P(t)$ (la justification)
- Mêmes mécanismes que la conjonction /\.

Le Quantificateur Existentiel (∃)
- Noté `exists x : T, P x` en Rocq.
- Constructivisme exige : il faut trouver *le témoin*.
- Preuve = Paire (t, π)
  - $t : T$ (le témoin)
  - $\pi : P(t)$ (la justification)
- Mêmes mécanismes que la conjonction /\.

En constructivisme strict, dire "Il existe", c'est être capable de montrer "Voici la chose". C'est pour ça qu'on a besoin d'exhiber le témoin physiquement avec la tactique ‘exists‘. Et quand on a une hypothèse existentielle, ‘destruct‘ nous la coupe en deux : le témoin, et sa preuve.

L'Égalité Propositionnelle (=)

Leibniz & Substitution

- $x = y$ est une proposition.

L'Égalité Propositionnelle (=)
Leibniz & Substitution
- $x = y$ est une proposition.

Venons-en à l'égalité. L'outil principal de l'algèbre à l'école, c'est de substituer y à la place de x dès qu'on sait que x = y. Rocq fait ça avec 'rewrite'. Remarquez que conceptuellement, cela invoque fondamentalement le principe de l'indiscernabilité des identiques de Leibniz. On s'en sert ici de manière utilitaire, en supposant que la plupart d'entre vous le maitrisent déjà un peu, et on justifiera la théorie profonde derrière ce principe la semaine prochaine.

L'Égalité Propositionnelle (=)

Leibniz & Substitution

- $x = y$ est une proposition.
- Indiscernabilité conceptuelle.

L'Égalité Propositionnelle (=)
Leibniz & Substitution
$x = y$ est une proposition.
Indiscernabilité conceptuelle.

Venons-en à l'égalité. L'outil principal de l'algèbre à l'école, c'est de substituer y à la place de x dès qu'on sait que x = y. Rocq fait ça avec 'rewrite'. Remarquez que conceptuellement, cela invoque fondamentalement le principe de l'indiscernabilité des identiques de Leibniz. On s'en sert ici de manière utilitaire, en supposant que la plupart d'entre vous le maitrisent déjà un peu, et on justifiera la théorie profonde derrière ce principe la semaine prochaine.

L'Égalité Propositionnelle (=)

Leibniz & Substitution

- $x = y$ est une proposition.
- Indiscernabilité conceptuelle.
- Tactique clé : `rewrite`

L'Égalité Propositionnelle (=)
Leibniz & Substitution
- $x = y$ est une proposition.
- Indiscernabilité conceptuelle.
- Tactique clé : `rewrite`

Venons-en à l'égalité. L'outil principal de l'algèbre à l'école, c'est de substituer y à la place de x dès qu'on sait que x = y. Rocq fait ça avec 'rewrite'. Remarquez que conceptuellement, cela invoque fondamentalement le principe de l'indiscernabilité des identiques de Leibniz. On s'en sert ici de manière utilitaire, en supposant que la plupart d'entre vous le maitrisent déjà un peu, et on justifiera la théorie profonde derrière ce principe la semaine prochaine.

Égalité Définitionnelle (≡) vs Propositionnelle (=)

- Propositionnelle : Exige une *preuve* manuelle (`rewrite`, lemmes).

Égalité Définitionnelle (≡) vs Propositionnelle (=)
- Propositionnelle : Exige une *preuve* manuelle (`rewrite`, lemmes).

C'est la chose la plus déroutante mais la plus puissante pour les nouveaux venus. Contrairement à la logique sur papier où vous devez justifier chaque étape de calcul mentalement, Rocq embarque l'exécution du code dans la théorie logique. La tactique reflexivity dit non seulement x=x, mais aussi que tout terme qui s'évalue à x est égal à x ! Ca permet des preuves magiques pour les calculs triviaux.

Égalité Définitionnelle ($\equiv$) vs Propositionnelle ($=$)

- Propositionnelle : Exige une *preuve* manuelle (`rewrite`, lemmes).
- Définitionnelle : La *machine* évalue le code pour vous.
- Exemples :
  - $2 + 2 \equiv 4$ (calcul)
  - `doubler 2` $\equiv 4$ (expansion/β-réduction)

Égalité Définitionnelle ($\equiv$) vs Propositionnelle ($=$)
Propositionnelle : Exige une *preuve* manuelle (rewrite, lemmes).
Définitionnelle : La *machine* évalue le code pour vous.
Exemples :
$2 + 2 \equiv 4$ (calcul)
doubler $2 \equiv 4$ (expansion/β-réduction)

C'est la chose la plus déroutante mais la plus puissante pour les nouveaux venus. Contrairement à la logique sur papier où vous devez justifier chaque étape de calcul mentalement, Rocq embarque l'exécution du code dans la théorie logique. La tactique reflexivity dit non seulement x=x, mais aussi que tout terme qui s'évalue à x est égal à x ! Ca permet des preuves magiques pour les calculs triviaux.

Égalité Définitionnelle ($\equiv$) vs Propositionnelle ($=$)

- Propositionnelle : Exige une *preuve* manuelle (`rewrite`, lemmes).
- Définitionnelle : La *machine* évalue le code pour vous.
- Exemples :
  - $2 + 2 \equiv 4$ (calcul)
  - `doubler 2` $\equiv 4$ (expansion/β-réduction)
- Tactique : `reflexivity` triomphe sans effort sur $\equiv$.

Égalité Définitionnelle ($\equiv$) vs Propositionnelle ($=$)
- Propositionnelle : Exige une *preuve* manuelle (`rewrite`, lemmes).
- Définitionnelle : La *machine* évalue le code pour vous.
- Exemples :
  - $2 + 2 \equiv 4$ (calcul)
  - `doubler 2` $\equiv 4$ (expansion/β-réduction)
- Tactique : `reflexivity` triomphe sans effort sur $\equiv$.

C'est la chose la plus déroutante mais la plus puissante pour les nouveaux venus. Contrairement à la logique sur papier où vous devez justifier chaque étape de calcul mentalement, Rocq embarque l'exécution du code dans la théorie logique. La tactique reflexivity dit non seulement x=x, mais aussi que tout terme qui s'évalue à x est égal à x ! Ca permet des preuves magiques pour les calculs triviaux.

Synthèse

- `forall` $\sim$ fonctions ($\rightarrow$) / tactique `intro`
- `exists` $\sim$ paires ($\wedge$) / tactique `destruct`
- `rewrite` pour la substitution équationnelle
- `reflexivity` pour laisser l'ordinateur calculer

Synthèse
- forall ~ fonctions (→) / tactique intro
- exists ~ paires (∧) / tactique destruct
- rewrite pour la substitution équationnelle
- reflexivity pour laisser l'ordinateur calculer

On va maintenant passer au Live Coding sur jsCoq. Je vais vous montrer tout ça en action sur de petits exemples avant de vous laisser attaquer le TP de cette leçon 3.